

ShopStream Cloud - Technical Architecture Specification

Section 1: Executive Summary & Project Purpose

ShopStream is a cloud-native, distributed e-commerce platform built for high-throughput retail operations.

The system provides scalable product browsing, secure user authentication, order placement, and payment settlement.

Section 1.1: Core Architecture Principles

The platform adopts an asynchronous microservices topology designed for horizontal autoscaling.

Frontend clients interact exclusively through an edge API Gateway which routes requests to internal domain services.

All services emit domain events onto a central messaging backbone to ensure decoupling.

Explicit Tech Stack Overview:

- Frontend: Next.js 14, React, Tailwind CSS
- Gateway: Kong API Gateway
- Microservices: Node.js (NestJS), Go (Gin), Python (FastAPI)
- Databases: MongoDB 7.0, PostgreSQL 16
- Caching & Events: Redis 7.2, RabbitMQ 3.12

ShopStream Cloud - System Architecture

Section 2: Microservices & Component Inventory

The platform consists of five primary software services:

1. Web Storefront (web-storefront)

Technology: Next.js 14 with TypeScript and Tailwind CSS.

Provides server-side rendered storefront catalog, shopping cart, and customer checkout portal.

Communicates with the API Gateway via HTTPS / REST.

2. API Gateway (api-gateway)

Technology: Kong Gateway with reverse proxy routing and JWT rate limiting.

Terminates public TLS traffic and authenticates customer sessions using Redis token validation.

Routes requests to internal microservices via internal gRPC and HTTP/2.

3. Authentication & User Service (auth-service)

Technology: Node.js with Express and TypeScript.

Handles customer registration, credential verification, and OAuth token issuance.

Persists customer profile data in the PostgreSQL Users table.

Exposes endpoints: POST /api/v1/auth/register, POST /api/v1/auth/login.

4. Product Catalog Service (catalog-service)

Technology: Go (Gin framework) optimized for high-read throughput.

Manages category hierarchies, search indexes, SKU pricing, and inventory quantities.

Reads and writes directly to MongoDB Document Store.

Exposes endpoints: GET /api/v1/products, GET /api/v1/products/:id.

5. Order Processing Service (order-service)

Technology: NestJS framework with TypeScript.

Coordinates transactional checkout workflows, cart validation, and order state machines.

Stores orders, line items, and invoice states in PostgreSQL.

Publishes 'order.created' events to the RabbitMQ Event Bus.

Exposes endpoint: POST /api/v1/orders.

ShopStream Cloud - Data Storage & Messaging

Section 3: Persistence Layer & Event Pipeline

1. Orders Database (orders-db)

Type: Relational Database (PostgreSQL 16).

Purpose: Strictly ACID-compliant storage for financial transactions, customer orders, and line items.

Utilizes read-replicas for historical order analytics.

2. Product Catalog Database (catalog-db)

Type: Document Store (MongoDB 7.0).

Purpose: Flexible schemaless storage for dynamic product attributes, variants, reviews, and dimensions.

3. Session & Cache Store (redis-cache)

Type: In-Memory Key-Value Store (Redis 7.2).

Purpose: Transient caching of product listings, shopping carts, and validated JWT revocation lists.

4. Asynchronous Event Bus (rabbitmq-broker)

Type: Message Broker (RabbitMQ).

Purpose: Reliable event distribution for decoupled background operations:

- Event 'order.created': Consumed by Payment Integration Service.
- Event 'payment.settled': Consumed by Notification Service to dispatch receipts.

Section 3.1: External Integrations

Stripe Payment Gateway: Accessed by Payment Service over external HTTPS for credit card tokenization.

Note: The documentation does not specify the search engine clustering mechanism (Elasticsearch or Algolia is not stated).

ShopStream Cloud - Data Flow Walkthrough

Section 4: Key Transactional Execution Flows

Flow 1: Customer Order Placement & Payment Settlement

Step 1: Web Storefront submits checkout payload to API Gateway via POST /api/v1/orders over HTTPS.

Step 2: API Gateway validates bearer JWT and forwards request to Order Processing Service via gRPC.

Step 3: Order Processing Service validates cart items with Catalog Service and persists pending order in Orders Database (PostgreSQL).

Step 4: Order Processing Service publishes 'order.created' message to RabbitMQ Event Bus.

Step 5: Payment Service consumes message from RabbitMQ, calls Stripe API via HTTPS, and settles payment.

Step 6: Payment Service publishes 'payment.settled' event; Notification Service dispatches confirmation email.

Section 4.1: Known Limitations & Architecture Gaps

1. Disaster recovery multi-region replication strategy is not specified in this document.
2. Search indexing mechanism for catalog items is omitted from documentation.
3. Cold storage archiving for legacy orders older than 5 years is not defined.